

assignment_0, Methods 3, 2026

2026-09-17

REMEMBER: In your portfolio, make sure to include code that can reproduce the answers requested in the exercises below (**MAKE A KNITTED VERSION**). If it does not KNIT, it cannot be part of your portfolio

Assignment 0: Using multilevel modelling to model hierarchical data

In this assignment we will be investigating the *politeness* dataset of Winter and Grawunder (2012) and apply basic methods of multilevel modelling.

Dataset

The dataset has been shared on GitHub, so make sure that the csv-file is on your current path. Otherwise you can supply the full path.

```
politeness <- read.csv('politeness.csv') # read in data
```

Exercises and objectives

The objectives of the exercises of this assignment are:

- 1) Learning to recognize hierarchical structures within datasets and describing them
- 2) Creating simple multilevel models and assessing their fitness
- 3) Working both with linear predictors and categorical predictors

REMEMBER: In your report, make sure to include code that can reproduce the answers requested in the exercises below, i.e. make a knitted version

REMEMBER: This assignment will be part of your final portfolio

Iva

Exercise A - describing the dataset and understanding the nature of its variables

- 1) Describe the dataset, such that someone who happened upon this dataset could understand the 7 variables and what they contain. This article, <https://www.sciencedirect.com/science/article/pii/S0095447012000666>, might help (see Table 1)
 - i. in this description, categorise the 7 variables into either nominal, ordinal, interval or ratio
 - ii. include number of observations and missing observations

iii. what do you think are the response variables, i.e. what could work as y in fitting models?

2) Do some analyses with *scenario* as an independent variable

- i. fit a multilevel model with *f0mn* as the dependent variable and *scenario* as the independent variable. Model different intercepts for each *subject*
- ii. now fit a similar model, but this time have *scenario* be a nominal variable
- iii. judging from the residual variance, (*sigma(model)*), what difference does it make whether you encode *scenario* as a categorical variable or as an integer variable? Why do you get the difference?
- iv. plot the model matrices, X , from the two models, using *image.plot* from the *fields* package; describe the two models, and describe the difference between the two kinds of models fitted from these plots

Dori

3) Which variables should be indicated as nominal/categorical?

- i. use *factor()* to explicitly change these variables into the nominal/categorical category

Florin

Exercise B - pooling - linear regression

1) Across *scenario*'s, Do a no-pooling model, a complete-pooling model and a partial-pooling model with *total_duration* as the dependent variable and *f0mn* as the independent variable. Model individual intercepts for each *scenario* for the partial pooling and the no-pooling models

- i. compare the residual variance between the three models - which results in explaining most variance?
- ii. for the partial-pooling model, add different intercepts for each *subject* and compare the residual variance in this model with the remaining three
- iii. how would you model a no-pooling model that took both *subject* and *scenario* into account?
- iv. each *subject* was exposed to each *scenario*; what is the effect of using the second-level effect ($1 + \text{subject:scenario}$) with the first-level effects already modeled?

Dori

2) Plot a model for each subject, ignoring scenarios for now $\text{total_duration} \sim \text{f0mn} + (1 | \text{subject})$ i. create a plot with a subplot for each subject. This plot should contain the observed values for the dependent values and the fitted lines for each subject, ignoring differences in *scenarios* ii. describe what the plots and the model reveal about the relationship between *total_duration* and *f0mn* iii. why do you think the slope is negative? (check the paper this is based on)

Alf

Exercise C - pooling - categorical predictors

1) Fit a partial-pooling model with the dependent variable *f0mn* and the independent variable *attitude*. For second-level effects, include intercepts for *subject* and *scenario*.

```
partial_model <- lme4::lmer(
  f0mn ~ attitude + (1|subject) + (1|scenario), # including both intercepts
  data = politeness
)
```

- i. explain what the first-level effects say about the effect of attitude on fundamental frequency, f_{0mn} , and explain what the two second level effects say about the variance around those group level effects

Answer: To know fixed effects we need to look at the coefficients of the model.

```
partial_model_coef <- coef(summary(partial_model)) #We get the coefficients

partial_model_coef #we load the coefficients
```

```
##              Estimate Std. Error  t value
## (Intercept) 203.78192   16.603592 12.27336
## attitudepol -14.71546    4.098249 -3.59067
```

We can from this data see that attitude does have an impact on the fundamental frequency with a beta value of -14.72. We know the lmer function orders categorical data alphabetically, making the informal attitude be the intercept. We can therefore say that the first-level effects say that a change in attitude from being informal to polite makes fundamental frequency drop by 14.72 Hz.

We now look at the random effects.

```
print(lme4::VarCorr(partial_model), comp = "Variance")
```

```
## Groups   Name      Variance
## subject (Intercept) 4033.65
## scenario (Intercept) 105.28
## Residual              883.14
```

We see that the variance of subjects lay around 4033 and 105 around scenario. So our data varies more between subjects than it does around the test scenario.

- ii. also plot the X and Z matrices using `image.plot()` (for the Z , you will have to apply `as.matrix()` as `image.plot()` does not handle sparse matrices). Explain what they mean

Answer: We will first get the values using the lme4 library

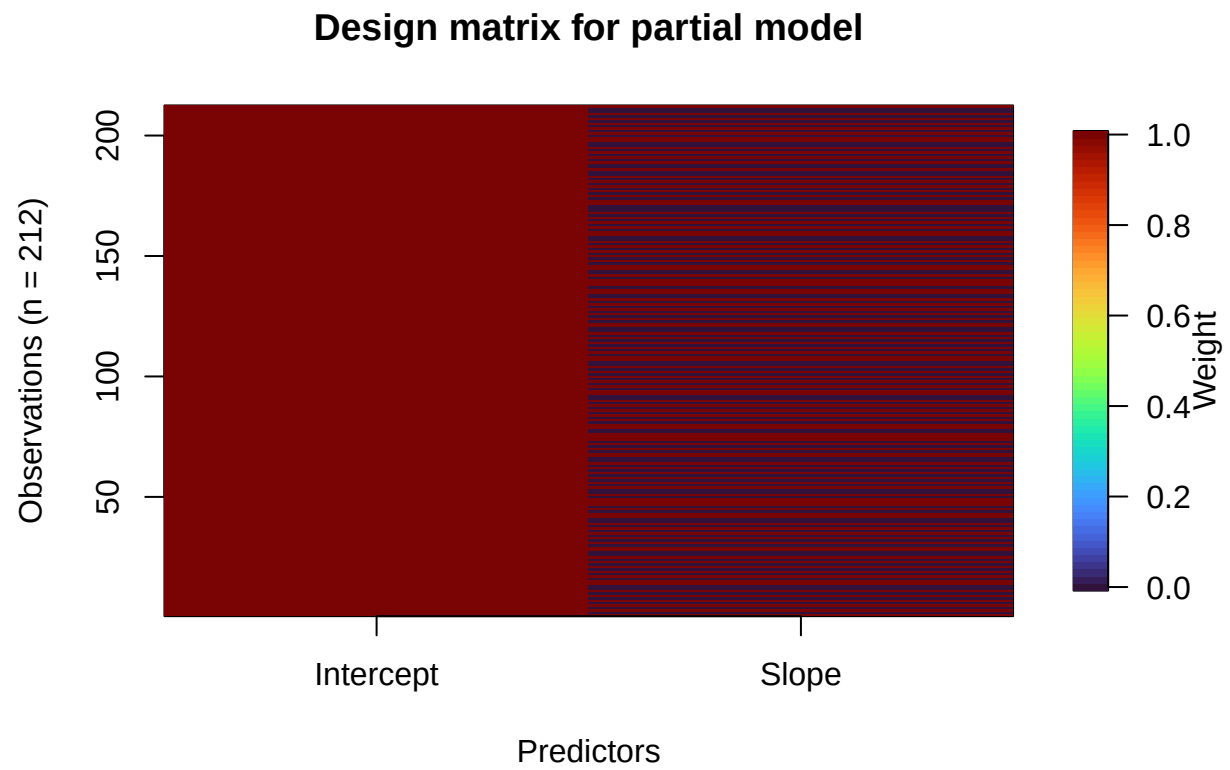
```
X <- lme4::getME(partial_model, "X")

Z <- lme4::getME(partial_model, "Z")

Z <- as.matrix(Z)

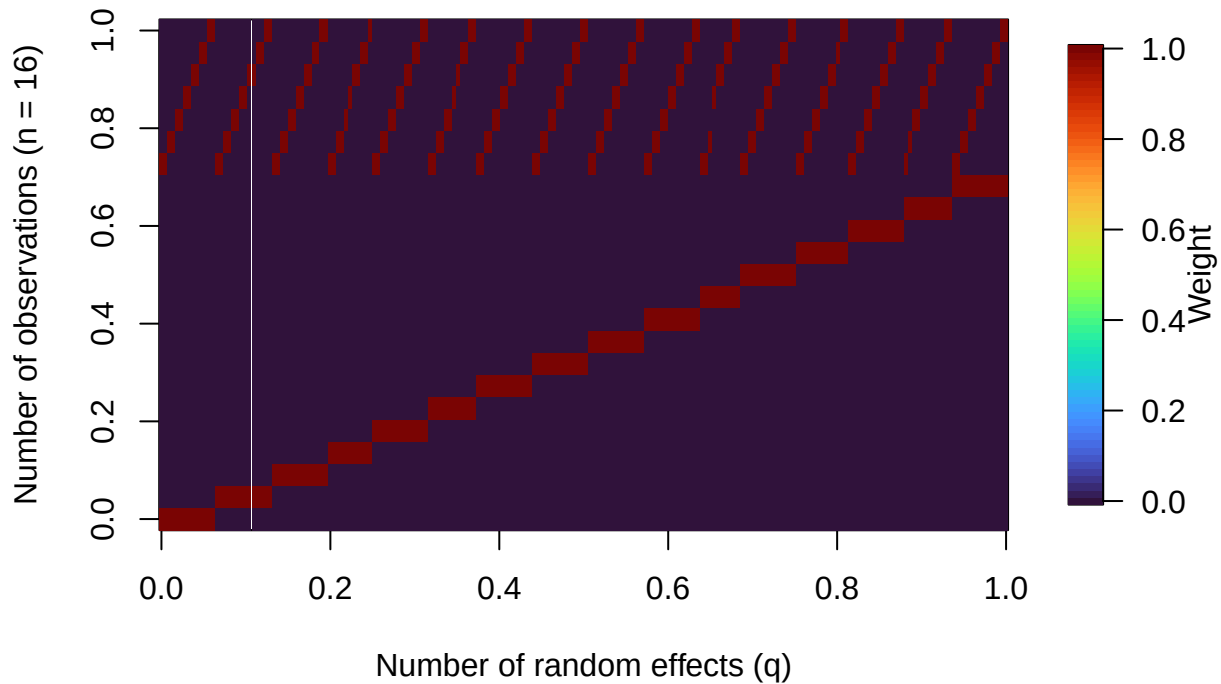
subjects <- politeness |> group_by(subject)

fields::image.plot(x = 1:dim(X)[2], y = 1:dim(X)[1], t(X), # Transposing to make the our
                  #design matrix look more traditional
                  xlab = "Predictors", ylab = "Observations (n = 212)", xaxt = 'n',
                  main = "Design matrix for partial model",
                  legend.lab = "Weight")
axis(1, 1:dim(X)[2], c('Intercept', 'Slope'))
```



```
fields::image.plot(Z,  
  xlab = "Number of random effects (q)", ylab = "Number of observations (n = 16)",  
  main = "Pattern of Z matrix",  
  legend.lab = "Weight")
```

Pattern of Z matrix



Our modelling of the X matrix shows us how our entire first column is made of 1s as intended with our second column made of the attitudes, either being 1 or 0 depending on if it is either a polite attitude or informal attitude.

The Z matrix holds two effects. It holds effects of both subject and scenario, as we modeled it to have. We can see how each subject, being each “step” of the big diagonal line” holds 1 values and all have a corresponding scenario represented through the diagonal - essentially being all the different scenarios that subject did.

- iii. fit an identical model, except that you remove *scenario* from the second-level effects. For both models, find the std. errors associated with the group-level effect of attitude - how do they differ and why?

Answer: We write code to find the std. errors to compare.

```
partial_model2 <- lme4::lmer(
  f0mn ~ attitude + (1|subject), # now only including one second-level effect
  data = politeness
)
print("Standard error of model with two second-level effects:")
```

```
## [1] "Standard error of model with two second-level effects:"
```

```
coef(summary(partial_model))[2,2]
```

```
## [1] 4.098249
```

```
print("Standard error of model with one second-level effects:")
```

```
## [1] "Standard error of model with one second-level effects:"
```

```
coef(summary(partial_model2))[2,2]
```

```
## [1] 4.311522
```

We see a higher standard error for the model with only one second-level effect. This is potentially because more second-level effects allow for more between-group variation to be explained, leaving less unexplained variance and therefore less standard error.

- 2) Making the second-level effects more complicated - now fit the two models with the formulas: $f0mn \sim \text{attitude} * \text{gender} + (1 | \text{subject}) + (1 | \text{scenario})$ and $f0mn \sim \text{attitude} * \text{gender} + (\text{gender} | \text{subject}) + (1 | \text{scenario})$

Answer: We are fitting a more complicated model.

```
complicated_model_1 <- lme4::lmer(f0mn ~ attitude * gender +  
                                (1 | subject) +  
                                (1 | scenario),  
                                data = politeness)  
  
complicated_model_2 <- lme4::lmer(f0mn ~ attitude * gender +  
                                (gender | subject) +  
                                (1 | scenario),  
                                data = politeness)
```

- i. Explain what each of the group-level effects and *subject* and *scenario* effects mean **Answer:** The group-level effects in the first complicated model are quite similar to the less complicated model. They model how the intercept changes according to both subject and the scenario. In our case, the subject effect is grouping the data by the individual participant's data and the scenario is grouping the data by what the scenario, or essentially what trial, the data was taken in.

The second complicated model has group-level random effect of gender and subject, which means we are modelling if the difference in gender varies from subject to subject. Do note that a subject in this dataset is only allowed one gender, making it a between-subject variable. Therefore, fitting the model to model how the effect of gender changes according to subject - thereby modelling a slope and intercept for each subject according to their different genders. But since there is only one gender per subject the gender slope is the same as just having a baseline, which only leaves the random intercept, and not really modelling any slope.

- ii. compare the within-group standard deviation, $\hat{\sigma}_y$ between the two models - why don't they differ? **Answer:** First we will access the group-level effects for each model.

```
print("The first complicated model")
```

```
## [1] "The first complicated model"
```

```
print(sigma(complicated_model_1))
```

```
## [1] 29.75684
```

```
print("The second complicated model")
```

```
## [1] "The second complicated model"
```

```
print(sigma(complicated_model_2))
```

```
## [1] 29.75729
```

We see here virtually no difference here. This clearly showcases the problem with modelling the random effect slope of genders of individual subject, by showing how it is not able to explain more variance even though we should be adding more complexity. Where the first model has random intercepts for both subjects and scenarios to explain more variation, the second model also models the difference in gender for each subject. But since each subject only has one layer this is essentially the same as only modelling the random intercepts - hence why we see not real difference in standard deviation.

- iii. fit this model `f0mn ~ attitude * gender + (gender | subject) + (gender | scenario)`; why does it give a singular fit? and why would you expect this given the design of the study?

```
singular_fit_model <- lme4::lmer(f0mn ~ attitude * gender +  
                                (gender | subject) +  
                                (gender | scenario),  
                                data = politeness)
```

```
## boundary (singular) fit: see help('isSingular')
```

```
lme4::isSingular(singular_fit_model) # checking if model is singular or near singular
```

```
## [1] TRUE
```

This model gives a singular fit due to the “dimensions” of the variance-covariance matrix have estimates of exactly zero. This makes sense since our study only allows for one gender per subject, as explained earlier, but doesn’t explain all of it since our earlier model was able to handle it. The last push towards our singular fit here might then be caused by the added `(gender | scenario)` random effect, since our scenario variable only holds 7 and we are now asking the model to compute the scenario-level intercepts, scenario-level slopes relative to gender, although it is possible, and their correlation - it turns into a lot of different parameters to be computed, but we only have 7 different scenarios to do it with.